



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

**Analisi di Sicurezza di Firmware IoT
Medicale
tramite Symbolic Execution**

Relatore: Prof. Danilo Mauro
Bruschi
Correlatore: Andrea Monzani

Elaborato Finale di:
Alessandro Mangili
24181A

ANNO ACCADEMICO 2025-2026

Indice

1	Introduzione	3
1.1	Sicurezza del Firmware nei Dispositivi Medicali	3
1.2	Obiettivi della tesi	3
1.3	Struttura della tesi	4
2	Background	5
2.1	Symbolic Execution	5
2.1.1	Introduzione alla Symbolic Execution	5
2.1.2	Fondamenti Teorici	7
2.1.3	Sfide e Limitazioni	8
2.1.4	Tecniche per Mitigare le Limitazioni	9
2.2	Strumenti e Tecnologie	9
2.2.1	Ghidra	9
2.2.2	LLVIM IR	10
2.2.3	RetDec	10
2.2.4	KLEE	11
2.2.5	Angr	11
3	Dispositivo Walk400h	13
3.1	Caratteristiche	13
3.2	Reverse Engineering del firmware	14
3.2.1	Estrazione e caratteristiche del binario	14
3.3	Funzioni critiche identificate	14
3.4	Meccanismi di monitoraggio implementati	16
4	Metodologia	17
4.1	Approccio RetDec + KLEE	17
4.1.1	Pipeline di decompilazione	17
4.1.2	Costruzione del modulo eseguibile	18
4.1.3	Stub delle periferiche e simbolizzazione mirata	19
4.2	Approccio Angr	20
4.2.1	Configurazione del progetto ed esecuzione Blind	20
4.2.2	Stub per l'esplorazione	21
4.2.3	Intercettazione UART	21
4.2.4	Modellazione di FatFS	24
4.2.5	Esplorazione percorsi	26
4.2.6	Breakpoint in memoria	26

5	Risultati	28
5.1	RetDec + Klee	28
5.1.1	Esiti dell'esecuzione	28
5.1.2	Decompilazione incompleta	28
5.2	Angr	29
5.2.1	Esplorazione dei percorsi	29
5.2.2	Breakpoint in memoria	31
5.2.3	La vulnerabilità dell'integer underflow	33
5.3	Vulnerabilità identificate	34
6	Conclusioni	36
6.1	Efficacia e limiti degli strumenti utilizzati	36
6.1.1	Limitazioni RetDec + KLEE	36
6.1.2	Efficacia e limiti dell'approccio con Angr	36
6.1.3	Sintesi comparativa	37
6.2	Impatto di sicurezza delle vulnerabilità identificate	37
6.2.1	Scenario di attacco combinato	37
6.3	Sviluppi futuri	38
6.4	Considerazioni finali	39

Capitolo 1

Introduzione

1.1 Sicurezza del Firmware nei Dispositivi Medicali

I dispositivi embedded medicali rappresentano ad oggi una categoria particolarmente critica dal punto di vista della sicurezza informatica. A differenza di dispositivi IT classici, questi operano in contesti in cui un malfunzionamento o una compromissione può tradursi in rischi per la salute dei pazienti o per la loro privacy [10].

La crescente connettività di questi apparati – tramite interfacce Bluetooth, Wi-Fi, USB o altri protocolli – ha ampliato enormemente i vettori di attacco possibili, esponendoli a minacce che in passato erano una prerogativa solo dei dispositivi classici quali personal computer e server. Come riportato dall’Agenzia per la Cybersicurezza Nazionale [2], nel periodo 2023–2025 sul territorio nazionale si sono verificati mediamente 4,7 eventi cyber malevoli al mese ai danni di strutture sanitarie, dei quali la metà ha avuto impatti sui servizi erogati e sulla privacy dei pazienti.

Per contrastare questi attacchi sono state proposte diverse soluzioni, ma le caratteristiche intrinseche di questi dispositivi rendono difficile l’utilizzo di strumenti di analisi pensati per sistemi standard. Ad esempio, i sorgenti sono raramente disponibili, e i binari risultanti sono spesso privi di simboli di debug, compilati per architetture non standard e progettati per operare in ambienti con scarse risorse hardware.

1.2 Obiettivi della tesi

L’obiettivo del progetto è quello di esplorare strumenti e metodologie per l’analisi di sicurezza di firmware embedded in dispositivi medicali, utilizzando come caso di studio il firmware dell’Holter Walk400h, un monitor cardiaco ambulatoriale rappresentativo della categoria di dispositivi medicali a bassa complessità ma con firmware non banale. A partire dal lavoro di reverse engineering condotto in una precedente tesi [3], che ha permesso di ricostruire la struttura del bootloader e di identificarne le funzioni critiche, l’obiettivo specifico di questo lavoro è valutare se strumenti di symbolic execution quali KLEE e angr permettano di individuare e validare concretamente una vulnerabilità nel meccanismo di aggiornamento del firmware, confrontandone al contempo l’efficacia e le limitazioni pratiche in un contesto di firmware bare-metal.

1.3 Struttura della tesi

Il resto della tesi è organizzato come segue. Il [Capitolo 2](#) introduce i concetti teorici alla base della symbolic execution, le relative sfide e le principali tecniche di mitigazione e presenta gli strumenti utilizzati nel lavoro: Ghidra, RetDec, KLEE e angr. Il [Capitolo 3](#) descrive il dispositivo Walk400h e il lavoro di reverse engineering del suo bootloader, individuando le funzioni critiche ai fini dell’analisi. Il [Capitolo 4](#) illustra la metodologia adottata, articolata nei due approcci esplorati – RetDec + KLEE e Angr. Il [Capitolo 5](#) presenta i risultati ottenuti con entrambi gli approcci e le vulnerabilità identificate. Il [Capitolo 6](#), infine, discute l’efficacia degli strumenti utilizzati, l’impatto di sicurezza delle vulnerabilità individuate, propone possibili sviluppi futuri e riassume le conclusioni del lavoro.

Capitolo 2

Background

2.1 Symbolic Execution

2.1.1 Introduzione alla Symbolic Execution

La symbolic execution è una tecnica di analisi dei programmi ibrida tra analisi statica e dinamica introdotta a metà degli anni '70, con l'obiettivo di verificare se determinate proprietà potessero essere violate da porzioni di codice. Gli aspetti di interesse tipici includono l'esecuzione di divisioni per zero, la dereferenziazione di puntatori nulli, e la presenza di backdoor o meccanismi che consentano di aggirare controlli di autenticazione.

L'esecuzione concreta di un programma impone all'esecutore di specificare valori ben definiti per tutte le variabili di input e proprio per questa caratteristica un solo percorso del codice sorgente potrà quindi essere esplorato. L'esecuzione simbolica supera questo limite permettendo di esplorare simultaneamente più percorsi, ovvero tutto l'insieme di istruzioni che verrebbero processate a fronte di input diversi. L'idea fondamentale dell'analisi simbolica è quindi quella di sostituire i valori concreti in input con valori simbolici, lasciando che il motore di esecuzione ragioni su tutti i possibili valori che questi potrebbero assumere.

Di seguito è riportato un frammento di pseudocodice, esempio classico in letteratura [6], con il relativo albero di esecuzione (Figura 2.1) che ne illustra il funzionamento sotto esecuzione simbolica.

```

int twice(int v) {
    return 2 * v;
}

void testme(int x, int y) {
    int z = twice(y);
    if (z == x) {
        if (x > y + 10)
            ERROR;
    }
}

/* simple driver exercising testme() with symbolic inputs */
int main() {
    int x = sym_input();
    int y = sym_input();
    testme(x, y);
    return 0;
}

```

Listing 2.1: Pseudocodice di esempio [6].

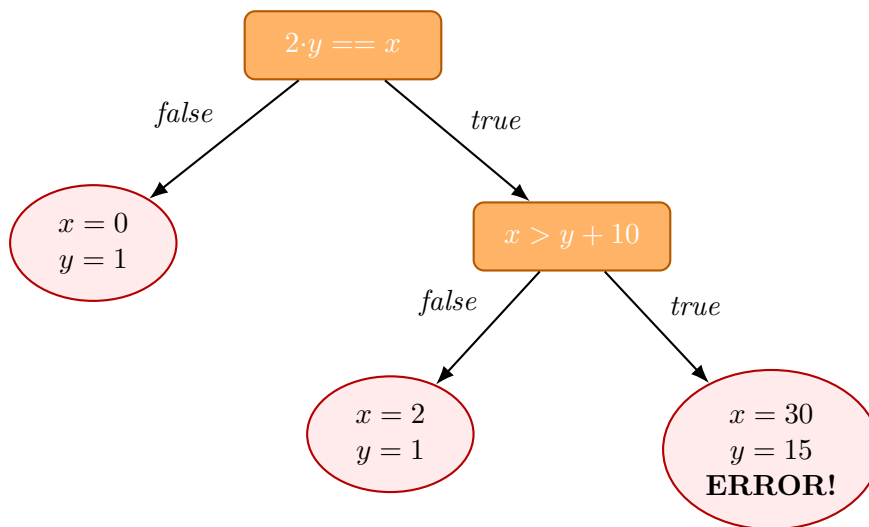


Figura 2.1: Execution tree per l'esempio in Figura 2.1.

L'albero di esecuzione corrispondente (Figura 2.1) mostra come l'engine di symbolic execution esplori entrambi i rami di ciascuna condizione, accumulando ad ogni diramazione un vincolo sulla path condition. Il nodo radice rappresenta la prima condizione, $z == x$, riscritta in termini di input simbolici come, $2y == x$; se la condizione è falsa il percorso termina in uno stato con $x = 0$, $y = 1$. Se invece è vera, l'esecuzione prosegue nel secondo controllo generando a sua volta due percorsi: uno che termina normalmente e uno che raggiunge lo stato di errore, individuando così automaticamente un input concreto in grado di far scattare la condizione di **ERROR**.

Questo semplice esempio evidenzia il valore pratico della symbolic execution rispetto al testing tradizionale: individuare manualmente la combinazione di input che innesci l'errore richiederebbe di risalire ai vincoli, mentre il motore simbolico li deriva automati-

camente esplorando l'albero delle possibilità.

La simulazione di un certo programma è quindi affidata a un motore di esecuzione simbolica, il quale mantiene, per ogni percorso esplorato, uno *stato* composto da:

- la memoria simbolica, che associa a ogni variabile un'espressione simbolica anziché un valore concreto;
- i path constraints, ovvero l'insieme dei vincoli logici accumulati lungo il percorso, che descrivono le condizioni sui valori simbolici necessarie affinché quel percorso venga raggiunto [5, 6].

2.1.2 Fondamenti Teorici

I concetti introdotti informalmente nell'esempio precedente possono ora essere formalizzati. La loro definizione segue un ordine logico preciso: lo stato descrive cosa il motore di esecuzione mantiene in ogni istante; le path condition descrivono i vincoli che tale stato accumula nel tempo; il solver SMT è lo strumento che permette di verificarne la soddisfacibilità; l'execution tree, infine, è la struttura che organizza l'insieme di tutti gli stati generati durante l'analisi.

Symbolic State Più formalmente, uno Stato Simbolico è composto dal program counter corrente, dal contenuto dei registri, dagli stack frame e dal contenuto della memoria. Questi ultimi tre elementi possono contenere una combinazione di valori concreti e variabili simboliche, insieme ai vincoli che li generano.

A partire da uno stato iniziale, il motore esegue il programma un'istruzione alla volta, aggiornando lo stato di conseguenza: i valori delle variabili concrete vengono aggiornati direttamente, mentre sulle variabili simboliche vengono aggiunti vincoli. È proprio l'insieme di questi vincoli, accumulati progressivamente lungo un percorso, a costituire la path condition di quello stato.

Path Conditions Le Path Conditions sono delle formule logiche che descrivono i vincoli sugli input simbolici affinché un determinato percorso venga raggiunto. A ciascun punto di diramazione, il motore genera nuovi stati figli, ognuno dei quali eredita la path condition del genitore estesa con il predicato che caratterizza il ramo intrapreso. Un percorso è considerato fattibile se e solo se la sua path condition è soddisfacibile, ovvero esiste almeno un'assegnazione concreta agli input simbolici che la rende vera [6].

Stabilire se una simile formula sia soddisfacibile non è però un problema che il motore di esecuzione simbolica risolve autonomamente: tale verifica viene delegata a un componente esterno specializzato, il solver SMT.

SMT Solver La verifica della soddisfacibilità delle path condition è delegata ad un SMT Solver, uno strumento formale in grado di determinare se una formula logica (Satisfiability Modulo Theories) ammette una soluzione. Nell'ambito della Symbolic Execution il solver SMT viene interrogato ad ogni diramazione per stabilire quali sono i rami percorribili. Tra i solver più diffusi ci sono Z3 e STP, i quali sono ampiamente integrati nei principali framework di analisi simbolica [1].

L'insieme delle decisioni prese dal solver a ogni diramazione, e degli stati che ne derivano, può essere rappresentato in una singola struttura che ne riassume visivamente l'evoluzione complessiva: l'execution tree.

Execution Tree L'Execution Tree è la struttura dati che rappresenta l'insieme di tutti i percorsi esplorati durante l'analisi. Ogni nodo dell'albero corrisponde ad uno

stato del programma (PC, valori concreti e simbolici, registri, path condition). Ogni arco rappresenta la transizione da uno stato al successivo, mentre i nodi foglia corrispondono a stati terminali: percorsi completati, errori rilevati o rami scartati [6].

2.1.3 Sfide e Limitazioni

La symbolic execution, pur essendo uno strumento potente per l'analisi automatica del software non è esente da problemi. Le principali sfide sono:

1. **Path explosion.** Una delle sfide principali della symbolic execution è la Path Explosion ossia la crescita esponenziale dei percorsi all'aumentare della complessità del programma.
2. **Gestione dei loop.** I programmi reali contengono frequentemente strutture iterative, talvolta con numeri di iterazioni indeterminati o potenzialmente infiniti. In questi casi l'engine rischia di non terminare senza convergere ad un risultato.
3. **Interazioni con ambiente esterno.** I programmi interagiscono con l'ambiente esterno attraverso chiamate di sistema, periferiche e altri sistemi hardware. Modellare correttamente tali interazioni è complesso poiché non sempre il comportamento esterno è ben noto o facilmente riproducibile.
4. **Memory modeling.** Accessi a puntatori e strutture dati dinamiche richiedono modelli accurati. In contesti embedded il problema si acuisce per la presenza di zone di memoria mappate su periferiche hardware [9].
5. **Dipendenza dall'architettura target.** L'engine deve riprodurre fedelmente la semantica dell'architettura su cui il programma è destinato ad essere eseguito (set di istruzioni, convenzioni di chiamata, endianness), che può differire significativamente da quella della macchina su cui l'analisi viene effettuata (ad esempio ARM rispetto a x86). Un modello architetturale impreciso o un disallineamento tra le due piattaforme può introdurre comportamento non rappresentativi del programma reale, o addirittura impedire l'esecuzione.

Queste sfide, per quanto formulate qui in termini generali, non pesano allo stesso modo su ogni classe di software. Nel contesto dei sistemi embedded e dei dispositivi IoT, in particolare, alcune di esse risultano sistematicamente aggravate: l'assenza di un sistema operativo completo espone direttamente il codice applicativo alle periferiche hardware, rendendo l'interazione con l'ambiente esterno (punto 3) non un caso limite ma la norma; allo stesso modo, l'assenza pressoché costante di documentazione pubblica su tali periferiche e sul layout di memoria del dispositivo rende il memory modeling (punto 4) e la fedeltà al modello architetturale (punto 5) ostacoli da affrontare fin dalle prime fasi dell'analisi. Questa combinazione di fattori è discussa in dettaglio da Qasem et al. [14], che nella loro survey distinguono esplicitamente le sfide generiche dell'analisi binaria da quelle proprie del dominio embedded, individuando proprio nella scarsità di informazioni sull'hardware sottostante e nei vincoli di risorse computazionali i fattori che più limitano l'applicabilità diretta delle tecniche di symbolic execution sviluppate per il software general-purpose. Nei capitoli successivi si vedrà come queste difficoltà si siano concretamente manifestate nell'analisi del firmware oggetto di questa tesi.

2.1.4 Tecniche per Mitigare le Limitazioni

1. **Concolic Execution.** La concolic execution è una tecnica che combina l'esecuzione concreta con quella simbolica. Questa tecnica mira a limitare il problema della path explosion, vincolando ogni singola esecuzione a un unico percorso concreto anziché forzare l'esplorazione simultanea di più stati in memoria. Un impiego rilevante di questo paradigma consiste nell'usarlo per modellare l'ambiente esterno al programma — per esempio delle periferiche hardware — quando questo risulta troppo complesso o indocumentato per essere rappresentato simbolicamente, delegando all'esecuzione concreta le porzioni di sistema più difficili da modellare [8]. Tuttavia, a differenza delle tecniche simboliche che esplorano più cammini simultaneamente condividendo lo stato in memoria, la concolic execution richiede una riesecuzione completa del programma per ogni nuovo percorso da analizzare: un costo che, se l'ambiente concretamente eseguito è un dispositivo fisico reale anziché un modello software, si moltiplica per il numero di cammini esplorati, potendo rendere questa tecnica più onerosa della sola symbolic execution [7].
2. **State Merging.** Il State Merging consiste nel fondere stati simbolici simili in uno più generico, riducendo il numero di stati attivi da esplorare. Per stati simbolici simili si intendono stati aventi lo stesso program counter – ossia giunti, lungo percorsi diversi, allo stesso punto nel programma. La fusione avviene introducendo per ciascuna variabile del programma un vincolo che ne discrimina il valore in base al percorso di provenienza. Sebbene questa tecnica comporti un aumento della complessità delle formule sottoposte al solver, consente di contenere l'esplosione degli stati in presenza di numerosi branch condizionali.
3. **State Pruning.** Il state pruning rileva gli stati simbolici già analizzati in precedenza e li scarta prima che vengano rieseguiti. Due stati sono considerati equivalenti se condividono lo stesso program counter e gli stessi valori per tutte le variabili simboliche. Questa tecnica è particolarmente efficace in presenza di loop e interrupt-driven code, come dimostrato da FIE, dove il pruning ha aumentato di sei volte il numero di analisi completate rispetto alla modalità senza ottimizzazioni [9].

2.2 Strumenti e Tecnologie

In questo capitolo vengono presentati gli strumenti utilizzati nel corso del lavoro di tesi. Per la realizzazione del progetto sono stati impiegati sia strumenti specifici per il reverse engineering del firmware, sia due diversi framework per l'analisi simbolica del binario, il cui utilizzo nelle varie fasi del progetto verrà descritto in dettaglio nel [Capitolo 4](#).

La fase di estrazione e ricostruzione del binario tramite reverse engineering è stata condotta in parte da un altro tesista, che ha permesso di ottenere le componenti principali del firmware del dispositivo medico. A partire da tali risultati, il presente lavoro si è concentrato sull'applicazione di tecniche di symbolic execution al binario estratto.

2.2.1 Ghidra

Ghidra è un framework di reverse engineering creato e mantenuto dalla **U.S. National Security Agency**. È uno strumento che permette a ricercatori di sicurezza, analisti di malware e sviluppatori di analizzare codice compilato per identificare e comprendere

le vulnerabilità di un dato software. Ghidra supporta una grande varietà di Instruction Set provenienti da architetture differenti (ARM, MIPS, PowerPC, e altre). Offre anche una funzionalità di decompilazione per mostrare all'utente dello pseudo-codice C/C++ a partire dal codice macchina grezzo. [13]

Nell'ambito di questo lavoro, Ghidra è stato impiegato esclusivamente in fase di analisi statica: caricamento del binario raw, disassemblato e decompilato sono stati utilizzati per ricostruire manualmente la struttura del bootloader (Sezione 3.2 e per identificare le funzioni critiche (Sezione 4.3). Il suo output è stato inoltre utilizzato come riferimento per valutare la fedeltà della decompilazione prodotta da RetDec (Sezione 6.1.2). Non sono state utilizzate le funzionalità di scripting o l'automazione tramite l'API Python/Java del tool.

2.2.2 LLVM IR

LLVM IR (Intermediate Representation) è il linguaggio intermedio alla base del progetto LLVM, un framework open-source per la costruzione di compilatori e strumenti di analisi del codice. LLVM IR si colloca a un livello di astrazione intermedio tra il codice sorgente ad alto livello e il codice macchina, ed è progettato per essere sufficientemente generico da rappresentare programmi provenienti da linguaggi e architetture differenti, pur mantenendo una struttura analizzabile e trasformabile in modo sistematico [12].

LLVM IR offre:

- una rappresentazione indipendente dall'architettura di destinazione, che consente di applicare le medesime analisi a binari compilati per piattaforme diverse;
- un formato sia testuale che binario (bitcode), interconvertibili tra loro, utile sia per l'ispezione manuale sia per l'elaborazione automatizzata;
- un ampio insieme di strumenti di trasformazione e ottimizzazione del codice, riutilizzabili anche al di fuori del contesto della compilazione tradizionale;
- un'ampia adozione come formato intermedio in numerosi strumenti di analisi binaria e symbolic execution, tra cui gli stessi framework impiegati in questo lavoro di tesi.

2.2.3 RetDec

RetDec è un tool open-source sviluppato da Avast che si occupa della traduzione/lifting di binari a codice intermedio LLVM IR. Il nome RetDec deriva da **Retargetable Decompiler** indicando come l'obiettivo del progetto sia quello di creare un decompilatore generico di binari. Questo tool supporta un gran numero di architetture e accetta in input file di diversi formati. Poichè è basato su LLVM fa sì che non sia legato a particolari architetture, sistemi operativi o formati di file eseguibili. [4]

RetDec, oltre che da decompilatore, offre anche funzionalità come:

- analisi statica
- estrattore di simboli di debug
- ricostruzione di istruzioni
- rilevamento e ricostruzione di funzioni e classi gerarchiche in C
- disassembler e altro.

2.2.4 KLEE

Klee è un framework di esecuzione simbolica che esplora automaticamente molteplici percorsi di esecuzione di un programma trattando gli input come valori simbolici piuttosto che concreti. Combina symbolic execution e constraint solving per generare casi di test con un'alta copertura del codice e trovare bug nel software. Klee esegue LLVM Bitcode permettendogli di analizzare programmi scritti in linguaggi che compilano in LLVM IR, come C/C++. Nel caso in cui il target di analisi sia invece un binario già compilato — come nel presente lavoro di tesi — è necessario un passaggio preliminare di lifting, che ricostruisca una rappresentazione in LLVM IR a partire dal codice macchina; a tale scopo, in questo lavoro è stato impiegato RetDec (Sezione 2.2.3), il cui output in LLVM IR costituisce l'input fornito a KLEE per l'analisi simbolica.

Le funzionalità principali di Klee sono:

- **generazione automatica di test-case ad alta copertura**, ottenuta forkando l'esecuzione ad ogni branch condizionato da un valore simbolico, così da massimizzare la percentuale di istruzioni e cammini del programma effettivamente esercitati;
- **gestione della memoria simbolica e operazioni complesse con puntatori**, come array indicizzati con valori simbolici o puntatori il cui target dipende dall'input, non trattabili con una semplice simulazione a valori concreti;
- **rileva vari errori di esecuzione**, accessi out-of-bounds, divisioni per zero, violazioni di asserzioni);
- **risoluzione dei vincoli**, per verificarne la soddisfacibilità;
- **riproduzione dei test case**, forniti in esecuzione reale, riproducono il medesimo percorso individuato durante l'analisi simbolica.

L'*engine di esecuzione simbolica* si occupa di interpretare le istruzioni LLVM Bitcode una ad una, tracciando valori simbolici e vincoli tra diversi cammini di esecuzione. Quando il risultato di una condizione dipende da un valore simbolico, l'engine esplora tutti i percorsi forkando lo stato di esecuzione.

Klee si interfaccia con diversi *constraint solver* come: STP, Z3, metaSMT e altri per determinare quale cammino di esecuzione sia fattibile generando gli input concreti che lo attraversano. Vengono inoltre fornite diverse euristiche di ricerca per decidere il prossimo cammino da esplorare:

- Depth-First Search (DFS)
- Breadth-First Search (BFS)
- Random Path
- Weighted Random
- Coverage-Optimized Search

KLEE mette a disposizione diversi strumenti da linea di comando: **klee** è il tool principale che prende in input LLVM bitcode ed esegue l'analisi simbolica; **kleaver** permette di interagire direttamente con il constraint solver; **klee-stats** visualizza le statistiche dell'esecuzione e **klee-replay** consente di rieseguire i test case per riprodurre i bug trovati. [11]

2.2.5 Angr

Angr è un framework open-source per l'analisi di binari, sviluppato dal Computer Security Lab dell'Università della California, Santa Barbara e da SEFCOM dell'Arizona

State University. È implementato come una suite di librerie Python 3 ed è indipendente dall'architettura target, supportando quindi binari compilati per diverse piattaforme. Le funzionalità principali offerte da Angr includono: esecuzione simbolica, analisi statica, disassembly e *lifting* a rappresentazione intermedia, analisi del flusso di controllo, analisi delle dipendenze dei dati, *Value-Set Analysis* (VSA) e decompilazione. A differenza di KLEE, che opera su codice sorgente compilato in LLVM Bitcode, Angr lavora direttamente sui binari, rendendolo particolarmente adatto a scenari in cui il codice sorgente non è disponibile. Internamente, Angr traduce le istruzioni binarie in una rappresentazione intermedia tramite VEX IR, sulla quale applica le proprie analisi simboliche e statiche.

Capitolo 3

Dispositivo Walk400h

3.1 Caratteristiche



Figura 3.1: Dispositivo Walk400h.

Il Walk400h è un dispositivo Holter prodotto da Cardioline S.p.a, azienda con sede a Trento, destinato alla registrazione ambulatoriale prolungata del segnale ECG, con una durata di registrazione fino a 7 giorni. Ai fini di questo lavoro non sono di interesse gli aspetti legati all'acquisizione clinica del segnale, quanto piuttosto l'architettura del sistema embedded che gestisce il dispositivo.

Dal punto di vista architetturale, il Walk400h è basato su un microcontrollore della famiglia **ARM Cortex-M4** (in particolare **STM32L471VGT6**), con un firmware eseguito in modalità **THUMB**. Il dispositivo integra una memoria Flash di 1 MB utilizzata per il firmware. Per quanto riguarda la memorizzazione dei dati del paziente e aggiornamenti del firmware viene invece usata una **scheda SD**.

Il Walk400h espone diversi canali di interazione con l'esterno che costituiscono, da un punto di vista della sicurezza, altrettante superfici di attacco potenziali: l'interfaccia **USB** per il trasferimento dei dati registrati verso il software di analisi, un'interfaccia **seriale UART** utilizzata per il logging interno e il meccanismo di aggiornamento basato su **scheda SD**, quest'ultimo di particolare interesse in quanto rappresenta il principale punto di ingresso di input non fidati nel sistema.

3.2 Reverse Engineering del firmware

Il lavoro di reverse engineering del bootloader del Walk400h è stato condotto da Alberici [3], il quale ha ricostruito la struttura del firmware del dispositivo e ne ha identificato la procedura di aggiornamento. In particolare, tale analisi ha evidenziato come questa procedura non implementi controlli adeguati sull'origine e sull'integrità del nuovo firmware caricato sul dispositivo, aprendo così a un potenziale vettore di attacco per l'esecuzione di codice non autorizzato. Il passo preliminare necessario a questo scopo è stata la ricostruzione dettagliata della struttura del bootloader del Walk400h, volta in particolare a individuare, all'interno del flusso di esecuzione, tutte le componenti del firmware legate al meccanismo di aggiornamento — così da delimitare con precisione le porzioni di codice da sottoporre successivamente a symbolic execution.

A partire da questo risultato, il presente lavoro si pone l'obiettivo di verificare concretamente lo sfruttabilità di tale mancanza di controlli, tramite l'applicazione di tecniche di symbolic execution alle porzioni di codice coinvolte nel processo di aggiornamento.

3.2.1 Estrazione e caratteristiche del binario

Il firmware è stato fornito come immagine binaria raw, priva di metadati (formato eseguibile, simboli di debug, ...). L'analisi statica è stata condotta con **Ghidra** che ha permesso di determinare le caratteristiche architetturali necessarie per una decompilazione corretta:

- **Architettura:** ARM Cortex-M4
- **Set di istruzioni:** Thumb, little-endian
- **Base di caricamento (VMA):** 0x08000000, coerente con il layout tipico della memoria Flash su microcontrollori di questa famiglia
- **Entry point del bootloader:** 0x08001e34

Questi parametri sono stati successivamente riutilizzati come parametri per la fase di decompilazione tramite RetDec, necessaria per produrre una rappresentazione LLVM IR del bootloader analizzabile da KLEE.

3.3 Funzioni critiche identificate

A partire dall'entry point, l'analisi del flusso di controllo ha permesso di ricostruire la logica principale del bootloader e di isolare un sottoinsieme di funzioni ritenute critiche per la sicurezza del dispositivo, in quanto ritenute responsabili della gestione dell'aggiornamento firmware e dell'accesso alla SD Card.

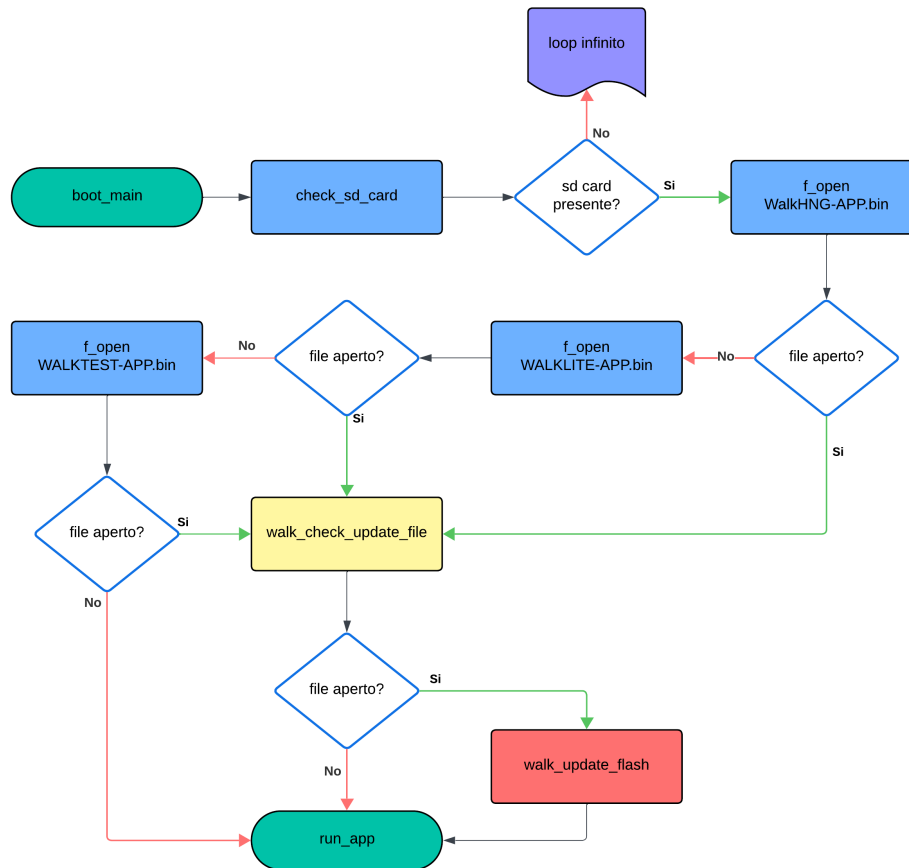


Figura 3.2: Flowchart semplificato delle funzioni chiave del bootloader del Walk400h.

boot_main punto d'ingresso della logica di avvio, responsabile dell'orchestrazione del processo di verifica e aggiornamento del firmware.

Criticità: componente centrale che decide se procedere con l'aggiornamento sulla base dell'esito di alcune verifiche; un errore di logica in questo punto può compromettere l'intero processo di boot.

walk_check_update_file verifica della validità del file di aggiornamento (WALKLITE-APP.bin) prima che il relativo contenuto venga scritto in memoria Flash.

Criticità: costituisce l'unico punto di controllo tra un file proveniente da un dispositivo di storage removibile e la scrittura in memoria persistente; una validazione insufficiente a questo livello permetterebbe la scrittura di un firmware non verificato.

walk_update_flash scrittura effettiva del nuovo firmware in memoria Flash.

Criticità: operazione privilegiata e irreversibile; un suo utilizzo scorretto (ad esempio raggiungibile bypassando i controlli di **walk_check_update_file**) comporterebbe la persistenza di codice malevolo sul dispositivo.

Di particolare interesse è la funzione **walk_check_update_file**, che costituisce l'unico punto di controllo del file di aggiornamento prima che questo venga considerato valido. Come mostrato in Figura ??, la funzione verifica dapprima l'esistenza del file e che lo spazio disponibile in Flash sia sufficiente per la sua dimensione dichiarata (**filinfo.size**); in caso contrario, restituisce **FR_INT_ERR**. Superati questi controlli, legge i primi 0x20 (32) byte del file in un buffer (**magic**) e ne verifica il

contenuto tramite `mod_strcmp` e il confronto di quattro byte in coda al buffer (offset `0x1c-0x1f`): solo se entrambe le condizioni sono soddisfatte il file viene considerato valido (`FR_OK`).

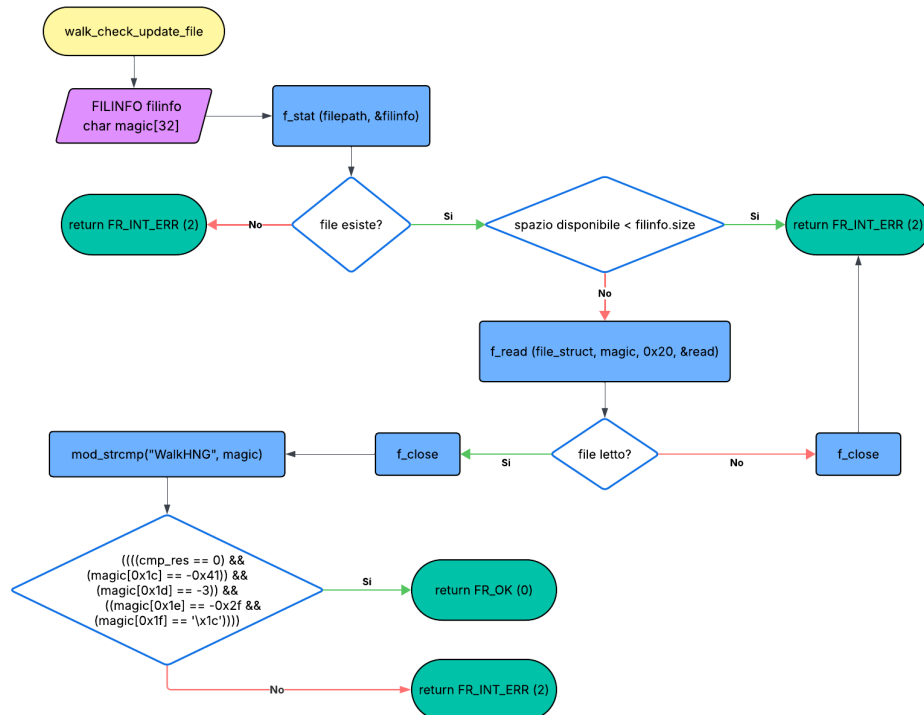


Figura 3.3: Flowchart semplificato della funzione `walk_check_update_file`.

3.4 Meccanismi di monitoraggio implementati

Nell’ottica di validare i risultati dell’analisi statica condotta in questa sezione e descritta nel lavoro di Alberici [3], e per supportare la successiva esplorazione simbolica del medesimo percorso con gli strumenti di analisi utilizzati, sono stati implementati due meccanismi di osservabilità del comportamento del firmware: il primo consente di monitorare i messaggi stampati via UART, il secondo di rilevare le scritture sulla periferica di memoria Flash. Entrambi i meccanismi sono descritti in seguito.

Cattura dei messaggi UART Il primo meccanismo ha l’obiettivo di monitorare i messaggi di logging emessi dal firmware via UART durante l’esecuzione simbolica: poiché tali messaggi descrivono in linguaggio naturale le fasi attraversate dal firmware (avvio, verifica della SD Card, validazione del file, ecc.), la loro cattura consente di ricostruire a posteriori quali rami decisionali siano stati attraversati lungo ciascun percorso esplorato, senza dover analizzare manualmente lo stato simbolico a ogni diramazione.

Monitoraggio della Flash Il secondo meccanismo ha l’obiettivo di rilevare il momento in cui il firmware tenta effettivamente una scrittura nella regione di Flash riservata all’applicazione: individuare questo istante è essenziale per poter osservare, e non solo dedurre indirettamente, se e quando il processo di aggiornamento del firmware giunga a compimento, superando l’intera catena di controlli descritta in precedenza.

Capitolo 4

Metodologia

4.1 Approccio RetDec + KLEE

4.1.1 Pipeline di decompilazione

Per poter applicare KLEE al firmware del Walk400h è stato necessario ottenere una rappresentazione in LLVM IR, formato su cui l'engine di symbolic execution opera nativamente. A tale scopo, il binario raw estratto ed analizzato nel punto 4.2 è stato passato in input a RetDec con i parametri architetturali individuati in fase di reverse engineering (modalità THUMB, entry point e indirizzo di caricamento), ottenendo come output il bitcode LLVM corrispondente.

Poichè RetDec non riusciva a completare la decompilazione dell'intero file binario del firmware – la cui dimensione originale è di 1 MB –, sono stati estratti solo i primi 256 KB. Questa strategia è giustificata dal layout standard della memoria flash ARM Cortex-M e verificata tramite l'analisi con Ghidra:

- **Vector table:** 0x08000000 - 0x080003FF, 1KB
- **codice eseguibile:** 0x08000400 - 0x08028200 (160KB)
- **dati statici:** resto del binario

Lo script per l'estrazione e la decompilazione è riportato di seguito.

```
1  set -e
2
3  INPUT="$(pwd)/input"
4  OUTPUT="$(pwd)/output"
5
6  dd if=../Firmware/walk400h.bin of="$INPUT"/fw_256k.bin bs=1
   ↪   count=$((0x40000))
7  chmod 777 "$INPUT"/fw_256k.bin
8
9  retdec-decompiler \
10   -a thumb -e little -m raw \
11   --raw-entry-point 0x08001e34 \
12   --raw-section-vma 0x08000000 \
13   "$INPUT"/fw_256k.bin
14
```

4.1.2 Costruzione del modulo eseguibile

Il bitcode prodotto da RetDec non è direttamente eseguibile da KLEE: il suo utilizzo ha richiesto la risoluzione di una serie di incompatibilità strutturali, riassunte in Tabella 4.1, individuate in modo incrementale a partire dagli errori restituiti da `llvm-as` e da KLEE stesso.

Problema	Sintomo	Soluzione
Variabili globali malformate	invalid use of function-local name in fase di assemblaggio	Normalizzazione delle dichiarazioni tramite sostituzione testuale mirata
Entry point mancante	KLEE: ERROR: Entry function 'main' not found	Creazione di un modulo wrapper con <code>main()</code> , che invoca <code>boot_main</code> con argomenti resi simbolici, linkato al bitcode del bootloader
Conflitti di target/data layout e simboli duplicati	Warning di linking e funzioni non risolte	Normalizzazione di target triple/datalayout; funzioni originali dichiarate a <i>weak linkage</i>
Variabile globale non caricabile	KLEE: ERROR: Unable to load symbol() while initializing globals	Ridefinizione della variabile da globale esterna a globale inizializzata a 0

Tabella 4.1: Problemi riscontrati nella costruzione del modulo eseguibile e relative soluzioni.

```

1  #include <stdint.h>
2  #include <string.h>
3
4  // =====
5  // KLEE symbolic execution interface
6  // =====
7
8  void klee_assert(int condition);
9  void klee_assume(int condition);
10 void klee_make_symbolic(void *addr, unsigned long nbytes, const char
    ↪ *name);
11 void klee_warning(const char *message);
12
13 // =====
14 // Firmware entry point declaration
15 // =====
16
17 extern int function_8007fb8(int arg1, int arg2);
18
19
20 int main(void) {

```

```

21     int arg1, arg2, arg3, arg4;
22
23     klee_make_symbolic(&arg1, sizeof(arg1), "arg1");
24     klee_make_symbolic(&arg2, sizeof(arg2), "arg2");
25
26     klee_assume(arg1 != 0);
27     klee_assume(arg2 != 0);
28
29     return function_8007fb8(arg1, arg2);
30 }

```

4.1.3 Stub delle periferiche e simbolizzazione mirata

Oltre ai conflitti strutturali descritti in [Sezione 4.1.2](#), l'esecuzione simbolica ha incontrato limitazioni legate all'assenza di un modello per le periferiche hardware e alla gestione delle variabili locali, riassunte in [Tabella 4.2](#).

Problema	Sintomo	Soluzione
Accessi hardware non modellabili	KLEE: ERROR: invalid pointer: klee_make_symbolic su registri MMIO	Sanitizzazione del bitcode: sostituzione dei load da indirizzi hardware e rimozione delle store corrispondenti
Variabile locale passata per riferimento	Wrong size given to klee_make_symbolic	Sostituzione, a livello di LLVM IR, del riferimento allo stack con una variabile globale dedicata (buffer magic)

Tabella 4.2: Problemi riscontrati nella simbolizzazione degli input e relative soluzioni.

Un vincolo più generale di KLEE, riscontrato nel tentativo di simbolizzare direttamente il buffer di destinazione all'interno dello stub di `f_read`, riguarda l'impossibilità di rendere simbolici indirizzi appartenenti allo spazio di memoria del programma stesso (ad esempio un puntatore a una variabile locale sullo stack) tramite una semplice chiamata a `klee_make_symbolic` sul puntatore ricevuto come parametro: il tentativo produceva un arresto immediato dell'esecuzione. Questo vincolo, sommato a quello sulle variabili locali passate per riferimento, ha reso necessaria la strategia di riscrittura manuale del bitcode descritta di seguito.

Le parti di codice riportate sotto mostrano il confronto tra il codice LLVM IR prima e dopo la riscrittura: il riferimento allo stack (`%stack_var_-40`), passato come secondo argomento alla chiamata a `f_read`, viene sostituito con la variabile globale `@klee_magic_buffer`.

```

dec_label_pc_8007e24:
%15 = call i32 @function_8007e38(i32 9792, i32* nonnull %stack_var_-40, i32 %14,
↳ i32* nonnull %stack_var_-76), !insn.addr !6891
%16 = call i32 @function_8007e76(i32 9792)
ret i32 2

```

```

dec_label_pc_8007c14:

```

```

%114 = call i32 @function_8007e38(i32 9792, ptr nonnull @klee_magic_buffer, i32 32,
↳ ptr nonnull %stack_var_-76), !insn.addr !1382
%115 = call i32 @function_8007874(i32 9792)
br label %common_ret

```

4.2 Approccio Angr

4.2.1 Configurazione del progetto ed esecuzione Blind

Il progetto Angr è stato configurato specificando l'architettura (ARMCortexM), l'indirizzo di caricamento (0x08000000) e l'entry point del bootloader individuato nella [Sezione 3.2](#), con il bit THUMB abilitato.

```

1  import angr, logging
2
3  THUMB = 1
4  BOOT_RESET_HANDLER = 0x08001e34
5
6  proj = angr.Project('../Firmware/walk400h.bin',
7                      arch='ARMCortexM',
8                      main_opts={'backend': 'blob', 'base_addr': 0x08000000,
9                                'entry_point': BOOT_RESET_HANDLER + THUMB}
10 )
11
12 state = proj.factory.entry_state(
13     add_options={
14         angr.options.ZERO_FILL_UNCONSTRAINED_REGISTERS,
15         angr.options.ZERO_FILL_UNCONSTRAINED_MEMORY,
16     }
17 )
18
19 simgr = proj.factory.simulation_manager(state)
20
21 while len(simgr.active) > 0:
22     try:
23         simgr.step(num_inst=1)
24     except Exception as e:
25         print(f'error on state {simgr.active}: {str(e)}')
26         simgr.move(from_stash='active', to_stash='_Drop')
27         print(f'\rsimgr: {simgr}, active: {simgr.active}, errored:
↳ {simgr.errored}, deadnoded: {simgr.deadended}', end="")

```

Un primo tentativo ha adottato un'esecuzione simbolica “blind”, priva di un target specifico: l'esplorazione avanza istruzione per istruzione su tutti i percorsi raggiungibili, terminando quando non restano più stati attivi. Questo approccio si è rivelato inefficiente, poichè l'assenza di un obiettivo non consente di indirizzare l'esplorazione verso le funzioni di interesse.

Dopo un certo numero di istruzioni, l'esecuzione risulta bloccata in un ciclo attorno all'indirizzo 0x08003165 (l'esecuzione oscilla tra più indirizzi vicini, riconducibili allo stesso ciclo).

```
1 [analysis] /workspace/angr \ $ python3 blind_exploration.py
2 simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x8003165>], errored:
  ↳ [], deadnoded: []
```

Oltre all'inefficienza, l'esecuzione blind si arresta dopo poche istruzioni: l'analisi dello stato mostra che il flusso rimane bloccato all'interno di cicli di attesa legati all'inizializzazione delle periferiche hardware, cicli che in ambiente simbolico non terminano mai poichè i registri hardware non vengono aggiornati da un controllore fisico esterno. Per questo problema sono state valutate due soluzioni:

- Modellazione simbolica delle periferiche: rendere simboliche zone di memoria MMIO, permettendo ad Angr di esplorare tutti i possibili stati dei registri
- Stub delle funzioni bloccanti: Sostituzione delle funzioni che dipendono dai cicli di attesa hardware con procedure che restituiscono immediatamente un valore fisso

4.2.2 Stub per l'esplorazione

Come anticipato in [Sezione 4.2.1](#), l'esecuzione simbolica "blind" non riesce a superare l'inizializzazione delle periferiche hardware, rimanendo bloccata in cicli di polling su registri non vincolati. Per procedere oltre queste fasi è stato quindi necessario introdurre degli **stub**: l'individuazione delle funzioni bloccanti ha richiesto l'analisi manuale del codice decompilato su Ghidra, mentre l'alternativa di rendere simbolica l'intera memoria delle periferiche è stata scartata per l'esplosione di stati che ne sarebbe derivata. Questo conferma come un approccio totalmente "blind" sia di fatto inapplicabile.

Per ridurre il numero di stati simbolici da esplorare e guidare l'esecuzione verso il flusso di aggiornamento del firmware, sono stati introdotti stub di due tipologie:

- **Stub di funzioni irrilevanti**: sostituiscono con un'istruzione vuota (**DoNothing**) le funzioni che non influenzano il flusso di aggiornamento del firmware, ma la cui esecuzione introdurrebbe stati superflui, come routine di inizializzazione.
- **Stub che ritornano un valore fisso**: sostituiscono funzioni il cui esito è necessario per proseguire lungo il percorso di interesse, forzando il valore di ritorno anzichè eseguire la logica reale - ad esempio `check_sd_card`, per garantire che la scheda SD risulti sempre presente, o le funzioni HAL di gestione della Flash (`HAL_FLASH_Unlock`, `HAL_FLASHEx_Erase`), le cui operazioni sulla memoria fisica non sono riproducibili in ambiente simbolico.

4.2.3 Intercettazione UART

Nell'ambito dell'esecuzione blind è stata inoltre condotta la cattura dei messaggi di debug a livello di registro, allo scopo di identificare quale periferica seriale fosse effettivamente utilizzata dal firmware. A partire dagli indirizzi delle periferiche USART/UART riportati nel datasheet del microcontrollore, sono stati impostati dei breakpoint di memoria sui rispettivi registri **TDR** (Transmit Data Register, ovvero a un offset di +0x28 rispetto all'indirizzo base di ciascuna periferica).

Come discusso in [Sezione 4.2.2](#), anche in questo caso è stato necessario applicare gli stub già individuati per permettere all'esecuzione di avanzare. Il codice sottostante è quello relativo alla cattura UART; sono riportate solamente le parti aggiuntive rispetto al codice base dell'esecuzione blind già mostrato in [Sezione 4.2.1](#).

```

1  import angr, logging
2  import claripy # type: ignore
3  from moduli.abs_hook import *
4
5  THUMB = 1
6  BOOT_RESET_HANDLER = 0x08001e34
7  BOOT_MAIN = 0x08007fb8
8
9  USART1 = 0x40013800
10 USART2 = 0x40004400
11 USART3 = 0x40004800
12 UART4 = 0x40004C00
13 UART5 = 0x40005000
14
15 def add_uart_breakpoints(s):
16     s.inspect.b('mem_write', mem_write_address=USART1 + 0x28,
17         ↪ when=angr.BP_AFTER, action=OnUsartTDRWrite)
18     s.inspect.b('mem_write', mem_write_address=USART2 + 0x28,
19         ↪ when=angr.BP_AFTER, action=OnUsartTDRWrite)
20     s.inspect.b('mem_write', mem_write_address=USART3 + 0x28,
21         ↪ when=angr.BP_AFTER, action=OnUsartTDRWrite)
22     s.inspect.b('mem_write', mem_write_address=UART4 + 0x28,
23         ↪ when=angr.BP_AFTER, action=OnUsartTDRWrite)
24     s.inspect.b('mem_write', mem_write_address=UART5 + 0x28,
25         ↪ when=angr.BP_AFTER, action=OnUsartTDRWrite)
26
27 def OnUsartTDRWrite(state):
28     addr_expr = state.inspect.mem_write_address
29     addr = state.solver.eval(addr_expr)
30
31     expr = state.inspect.mem_write_expr
32     conc_data = state.solver.eval(expr)
33     data = conc_data & 0xFF
34     print(f"\t{hex(addr)} -> {chr(data)}")
35
36 for addr in [
37     0x08001ef8, # HAL_Init
38     0x08002c04, # HAL_PWREx_EnableBatteryCharging
39     0x08002b64, # HAL_PWR_EnableBkUpAccess
40     0x080006e4, # walk_clocks_setup
41     0x0800084c, # walk_gpio_setup
42     0x08000b00, # walk_gpio_setup_2

```

```

53     0x080007c0, # walk_gpio_setup_3
54 ]:
55     proj.hook(addr + 1, DoNothing())
56
57     proj.hook(0x080053f8 + THUMB, ReturnZero()) #
    ↪     UART_WaitOnFlagUntilTimeout
58
59     simgr = proj.factory.simulation_manager(state)

```

Output:

```

1  simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x80054e1>], errored:
    ↪     [], deadnoded: [] 0x40004828 -> W
2  simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x80054e1>], errored:
    ↪     [], deadnoded: [] 0x40004828 -> a
3  simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x80054e1>], errored:
    ↪     [], deadnoded: [] 0x40004828 -> 1
4  [...]
5  simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x80054e1>], errored:
    ↪     [], deadnoded: [] 0x40004828 -> 0
6  simgr: <SimulationManager with 1 active>, active: [<SimState @ 0x80054e1>], errored:
    ↪     [], deadnoded: [] 0x40004828 -> 6

```

L'intercettazione a livello di registro si è rivelata funzionante - il breakpoint sui registri TDR ha permesso di identificare USART3 come periferica di logging - ma computazionalmente onerosa: il tempo di esecuzione cresce rapidamente all'aumentare del numero di stati simbolici attivi, rendendola poco praticabile su esplorazioni più estese. A questo scopo, nelle analisi successive, si è preferito adottare la cattura dei messaggi di debug con un meccanismo basato su hooking della funzione `printf`, meno oneroso in termini prestazionali.

Oltre all'hooking, è stato aggiunto un meccanismo di logging dei messaggi di debug: questo meccanismo utilizza la funzionalità dei plugin di Angr. I plugin sono delle estensioni dello stato che permettono di memorizzare dati custom durante l'esecuzione simbolica e vengono automaticamente duplicati quando si forka lo stato, garantendo che ogni branch mantenga la propria copia indipendente dei messaggi. È possibile registrare un nuovo plugin attraverso il comando:

```

1  state.register_plugin('printf_log', PrintfLogger())

```

La classe `HookPrintf` si occupa dell'hooking della funzione `printf` e svolge i seguenti compiti:

- intercetta le chiamate a `printf` durante l'esecuzione simbolica;
- estrae la format string e i primi tre argomenti, letti dai registri `r1`, `r2` e `r3` secondo la calling convention ARM;
- gestisce i format specifier più comuni (`%s`, `%d`, `%u`, `%x`);
- nel caso di `%x`, converte il valore in carattere ASCII se stampabile, altrimenti lo riporta in formato esadecimale;
- salva il messaggio formattato nel plugin `PrintfLogger`.

4.2.4 Modellazione di FatFS

Per modellare correttamente il flusso di aggiornamento del firmware è necessario poter interagire con le funzioni che accedono al filesystem sulla SD Card, in particolare con le strutture dati che queste manipolano. Le funzioni di libreria FatFS operano infatti su strutture dati con un layout preciso (`FIL`, `FILINFO`), i cui campi vengono letti e scritti dal firmware in punti diversi del flusso di validazione. Per poter riferire questi campi per nome anziché calcolarne manualmente l'offset in memoria, le definizioni di questo tipo di FatFS sono state registrate in angr tramite `angr.types.register_types`, fornendo un equivalente dei typedef originali.

```
4 ALL_TYPES_STRING = '''
5 typedef unsigned char BYTE;
6 typedef unsigned short WORD;
7 typedef unsigned int UINT;
8 typedef unsigned int DWORD;
9 typedef unsigned int FSIZE_t;
10 typedef unsigned int LBA_t;
11
12 typedef struct _FILINFO{
13     FSIZE_t fsize;
14     WORD    fdate;
15     WORD    ftime;
16     BYTE    fattrib;
17     char    fname[13];
18 } FILINFO;
19
20 typedef struct {
21     DWORD fs;          /* Pointer to the volume holds this object */
22     WORD  id;          /* Volume mount ID when this object was opened
23     ↪ */
24     BYTE  attr;        /* Object attribute */
25     BYTE  stat;        /* Object chain status (exFAT: b1-0: =0:not
26     ↪ contiguous, =2:contiguous, =3:fragmented in this session,
27     ↪ b2:sub-directory stretched) */
28     DWORD sclust;      /* Object data cluster (0:no data or root
29     ↪ directory) */
30     FSIZE_t objsize;    /* Object size (valid when sclust != 0) */
31     UINT  lockid;      /* File lock ID origin from 1 (index of file
32     ↪ semaphore table Files[]) */
33 } FFOBJID;
34
35 typedef struct {
36     FFOBJID obj;        /* Object identifier */
37     BYTE  flag;         /* File status flags */
38     BYTE  err;          /* Abort flag (error code) */
39     FSIZE_t fptr;       /* File read/write pointer (byte offset origin
40     ↪ from top of the file; 0 on open) */
41 }
```

```

35     DWORD    clust;          /* Current cluster of fptr (one cluster behind
    ↪ if fptr is on the cluster boundary; invalid if fptr == 0) */
36     LBA_t    sect;          /* Current data sector (can be invalid if fptr
    ↪ is on the cluster boundary)*/
37     LBA_t    dir_sect;      /* Sector number containing the directory
    ↪ entry */
38     BYTE*    dir_ptr;       /* Pointer to the directory entry in the
    ↪ window */
39     DWORD*    cltbl;        /* Pointer to the cluster link map table
    ↪ (nulled on file open; set by application) */
40     BYTE     buf[512]; /* File private data transfer buffer (Always valid
    ↪ if fptr is not on the sector boundary but can be invalid if fptr
    ↪ is on the sector boundary.) */
41 } FIL;
42
43 '''
44
45 angr.types.register_types(angr.types.parse_types(ALL_TYPES_STRING))

```

Su questa base sono stati poi implementati gli hook HookFRead e HookFStat, che costituiscono l'esecuzione reale delle rispettive funzioni FatFS.

```

1  class HookFRead(Angr.SimProcedure):
2      def run(self, fil, buf, count, read_ptr):
3          ret_addr = hex(self.state.callstack.ret_addr)
4          position = self.state.solver.eval(self.state.mem[fil].FIL.fptr.resolved)
5          real_count = self.state.solver.eval(count)
6
7          fill_info_obj =
8              ↪ claripy.BVS(f'f_read_{hex(position)}_{hex(real_count)}_{ret_addr}',
8              ↪ real_count * self.state.arch.bits)
9          self.state.memory.store(buf, fill_info_obj, real_count, disable_actions=True,
9              ↪ inspect=False)
10         self.state.memory.store(read_ptr, real_count, 4, disable_actions=True,
10             ↪ inspect=False)
11
12         self.state.mem[fil].FIL.fptr = position + real_count
13
14         return 0
15
16 class HookFStat(Angr.SimProcedure):
17     def run(self, file_path, fill_info):
18         size = self.state.mem.FILINFO._type.size // 8
19         ret_addr = hex(self.state.callstack.ret_addr)
20         fill_info_obj = claripy.BVS(f'f_stat_{file_path}_{ret_addr}', size *
20             ↪ self.state.arch.bits)
21         self.state.memory.store(fill_info, fill_info_obj, size, disable_actions=True,
21             ↪ inspect=False)
22         return 0

```

- HookFRead non modella l'accesso reale al filesystem sulla SD Card, ma scrive nel buffer di destinazione (buf) una variabile simbolica di dimensione pari alla quantità richiesta (count), aggiornando contestualmente il campo fptr della struttura FIL

e il numero di byte letti (`read_ptr`). Il nome della variabile simbolica contiene posizione, quantità richiesta e indirizzo di ritorno della chiamata, garantendo che letture diverse generino variabili indipendenti anzichè condividere lo stesso valore simbolico.

- `HookFStat` rende simbolica l'intera struttura di `FILINFO` restituita al chiamante

Questo approccio permette al solver di determinare automaticamente quali valori concreti di questi campi soddisfano i vincoli imposti dalla logica di validazione del firmware.

4.2.5 Esplorazione percorsi

Per rendere l'esplorazione mirata anzichè esaustiva, è stata adottata la funzione `simgr.explore()` di Angr, configurabile tramite tre parametri principali:

- un **target**: l'indirizzo che l'esecuzione deve raggiungere per considerarsi conclusa con successo;
- uno o più indirizzi da **evitare**, che vengono scartati non appena raggiunti;
- un numero massimo di step, oltre il quale l'esplorazione si interrompe anche in assenza di stati che soddisfino la condizione di target.

Come target è stato scelto l'indirizzo di `walk_update_flash`, funzione responsabile della scrittura del firmware in memoria Flash, mentre come indirizzo da evitare è stato specificato `run_app`, corrispondente all'avvio dell'applicativo principale: questo evita che l'esecuzione prosegua oltre il bootloader una volta completato il flusso di aggiornamento.

```
94 simgr = proj.factory.simulation_manager(state)
95 simgr.explore(
96     find=WALK_FLASH_UPDATE + THUMB,
97     avoid=[
98         RUN_APP + THUMB
99     ],
100     n=5000
101 )
```

L'esplorazione condivide con l'esecuzione blind gli stub per le funzioni di inizializzazione e utilizza l'hook di `printf` descritto in [Sezione 4.2.3](#). A questi si aggiungono gli stub mirati a guidare il flusso di esecuzione verso `walk_update_flash`, insieme agli hook per la modellazione di FatFS descritti in [Sezione 4.2.4](#). In questo modo, se l'esplorazione raggiunge il target, il solver di Angr può essere interrogato per determinare quali valori delle variabili rese simboliche soddisfano i vincoli imposti dalla funzione `walk_check_update_file`.

4.2.6 Breakpoint in memoria

Il secondo approccio target-oriented adottato è basato su un breakpoint in memoria e condivide gran parte della struttura con il metodo precentente: la configurazione del progetto, gli stub e la modellazione di FatFS restano invariati. La differenza risiede nel criterio con cui l'esplorazione viene considerata conclusa.

Mentre l'esplorazione termina quando si raggiunge un determinato indirizzo, l'approccio a breakpoint termina quando si verifica quando viene rilevata una scrittura nella regione di Flash riservata all'applicazione. A tale scopo viene registrato un breakpoint di inspection sull'evento di scrittura in memoria, filtrato sull'indirizzo di tale regione (FLASH_BASE, 0x0800e000); alla prima scrittura rilevata, il relativo handler imposta un flag nello stato corrente, che diventa la condizione di terminazione passata a `simgr.explore()`.

```
97 state.inspect.b('mem_write', mem_write_address = FLASH_BASE,  
    ↪ when=angr.BP_BEFORE, action=flash_write_handler)  
98 state.register_plugin('printf_log', PrintfLogger())  
99  
100 simgr = proj.factory.simulation_manager(state)  
101  
102 simgr.explore(  
103     find= lambda s: 'flash_write' in s.globals,  
104     avoid = [RUN_APP + THUMB ],  
105     n=5000  
106 )
```

Capitolo 5

Risultati

5.1 RetDec + Klee

L'esecuzione simbolica della pipeline RetDec + Klee non ha prodotto risultati significativi per quanti riguarda l'analisi, mettendo in luce le difficoltà di questi strumenti per analizzare un firmware embedded reale.

5.1.1 Esiti dell'esecuzione

L'esecuzione con KLEE è terminata dopo 277 istruzioni, con un solo percorso esplorato in modo parziale e nessun percorso completato. La terminazione è avvenuta in corrispondenza di un accesso a puntatore non valido (`null page access`), senza che l'esecuzione raggiungesse la logica di validazione del *magic number* all'interno di `walk_check_update_file`. Nonostante ciò l'output mostra che la pipeline è comunque riuscita ad avanzare fino all'ingresso di `walk_check_update_file`, superando le fasi di inizializzazione grazie agli stub creati.

5.1.2 Decompilazione incompleta

L'analisi della causa del mancato raggiungimento della logica di controllo del *magic number* ha evidenziato il problema principale di questo approccio: il codice prodotto da Retdec non è funzionalmente equivalente al binario originale. Confrontando il decompilato (nelle sue forme pseudo-C e LLVM IR) con il disassemblato Ghidra emergono infatti delle omissioni significative.

In particolare, all'interno di `walk_check_update_file` manca completamente il ramo di validazione *magic number*. La mancanza di questo ramo impedisce a KLEE di esplorare i percorsi che attraversano la validazione del file di aggiornamento, ossia proprio la logica di interesse per l'analisi.

Non è possibile escludere che l'output di RetDec presenti ulteriori omissioni analoghe per altre porzioni di codice.

Decompilato Ghidra (il ramo di validazione, con `mod_strcmp`, `strcpy`, ...)

```
1  if (res == FR_OK) {
2      f_close(&file_struct);
3      magic[7] = '\0';
4      cmp_res = mod_strcmp("WalkHNG", magic);
5      if (((cmp_res == 0) && (magic[0x1c] == -0x41)) && (magic[0x1d] == -3)) &&
```

```

6      ((magic[0x1e] == -0x2f && (magic[0x1f] == '\x1c')))) {
7      magic[0xf] = '\0';
8      magic[0x1b] = '\0';
9      strcpy(&global_magic, magic + 8);
10     strcat(&global_magic, " v");
11     strcat(&global_magic, magic + 0x10);
12     putc_state = '\0';
13     printf("---> Load File: %s\r", filinfo.fname);
14     printf("---> size: %d bytes\r", filinfo.fsize);
15     printf("---> Code: %s\r", &global_magic);
16     return FR_OK;
17 }
18 }

```

Decompilato RetDec (la stessa funzione: dopo `f_read` salta direttamente a `f_close` e `return 2`, il ramo di validazione non c'è):

```

1  int32_t walk_check_update_file(void) {
2      // 0x8007be4
3      int32_t v1; // bp-72, 0x8007be4
4      memset(&v1, 0, 32);
5      int32_t v2 = 0; // bp-76, 0x8007bf4
6      if (f_stat(*(int32_t *)0x20002400, &v1) != 0) {
7          // 0x8007cb8
8          return 2;
9      }
10     uint16_t v3 = *(int16_t *)0x1fff75e0; // 0x8007c06
11     if (v3 == -1) {
12         __asm_it();
13     } else {
14         __asm_itttt();
15     }
16     if ((v3 == -1 ? 0xf2000 : (1024 * (int32_t)v3 & 0x3ffc00) - 0xe000) < v1) {
17         // 0x8007cb8
18         return 3;
19     }
20     // 0x8007c24
21     int32_t v4; // bp-40, 0x8007be4
22     f_read(0x2640, &v4, 32, &v2);
23     f_close(0x2640);
24     return 2;
25 }

```

5.2 Angr

5.2.1 Esplorazione dei percorsi

L'esplorazione dei percorsi ha raggiunto con successo la funzione target `walk_update_flash`: il *simulation manager* riporta uno stato in `found`, a fronte di 95 stati ancora attivi e 12 scartati perchè diretti verso l'indirizzo da evitare.

Struttura del magic number Interrogando il solver sui vincoli accumulati lungo il percorso, è possibile ricostruire la sequenza di byte che il file di aggiornamento deve

contenere per superare la validazione. I 14 vincoli individuati si traducono nella seguente struttura dell'header:

Offset (byte)	Valore	Interpretazione
0-6	57 61 6c 6b 48 4e 47	"WalkHNG"
8, 16	00	padding
28-31	bf fd d1 1c	fine magic

Tabella 5.1: Decodifica dei vincoli sul buffer letto da `f_read`.

Si osserva, dal disassemblato Ghidra, che solo i primi 7 byte (il magic "WalkHNG") e gli ultimi 4 byte (bf fd d1 1c) sono effettivamente vincolanti ai fini della validazione; i byte del padding possono assumere un valore qualsiasi eccetto per il primo e l'ultimo che sono vincolati.

Il primo vincolo coinvolge invece la variabile simbolica creata in `HookFStat`, corrispondente all'intera struttura `FILINFO`: la porzione selezionata (`[703:672]`) isola il campo `fsize`, mentre `Reverse(...)` corregge l'ordine dei byte per l'endianness little-endian del microcontrollore. Il vincolo lega `filinfo.fsize` al valore di `available` calcolato dal firmware: per lo stato trovato, il solver ammette valori di `filinfo.fsize` fino a `0xffff2000` (~4 GB), coerentemente con la vulnerabilità di integer underflow discussa in Sezione 6.2.3.

```

1 Total constraints: 14
2
3 [ 1] <Bool 0xffff2000 >= Reverse(f_stat_<BV32
   ↳ 0x800bcd2>_0x8007c01_965_704[703:672])>
4 [ 2] <Bool f_read_0x0_0x20_0x8007c31_966_1024[1023:1016] == 87>
5 [ 3] <Bool f_read_0x0_0x20_0x8007c31_966_1024[1015:1008] == 97>
6 [ 4] <Bool f_read_0x0_0x20_0x8007c31_966_1024[1007:1000] == 108>
7 [ 5] <Bool f_read_0x0_0x20_0x8007c31_966_1024[999:992] == 107>
8 [ 6] <Bool f_read_0x0_0x20_0x8007c31_966_1024[991:984] == 72>
9 [ 7] <Bool f_read_0x0_0x20_0x8007c31_966_1024[983:976] == 78>
10 [ 8] <Bool f_read_0x0_0x20_0x8007c31_966_1024[975:968] == 71>
11 [ 9] <Bool f_read_0x0_0x20_0x8007c31_966_1024[799:792] == 191>
12 [10] <Bool f_read_0x0_0x20_0x8007c31_966_1024[791:784] == 253>
13 [11] <Bool f_read_0x0_0x20_0x8007c31_966_1024[783:776] == 209>
14 [12] <Bool f_read_0x0_0x20_0x8007c31_966_1024[775:768] == 28>
15 [13] <Bool f_read_0x0_0x20_0x8007c31_966_1024[959:952] == 0>
16 [14] <Bool f_read_0x0_0x20_0x8007c31_966_1024[895:888] == 0>

```

Call Stack

```

1 0 | boot_main -> walk_update_flash, returning to boot_main
2 1 | 0x0 -> 0x0, returning to 0x0

```

Execution Path A differenza dell'approccio RetDec + KLEE (Sezione 6.1), compare qui il branch di validazione del magic number, con le chiamate a `mod_strcmp`, `strcpy` e `strcat`.

```

1 boot_reset_handler
2 +-- boot_main

```

```

3      +--- HAL_Init
4      +--- walk_uart_setup
5      +--- check_sd_card
6      +--- f_open
7      +--- walk_check_update_file
8      |   +--- f_stat
9      |   +--- f_read
10     |   +--- mod_strcmp
11     |   +--- strcpy
12     |   +--- strcat
13     |   [...]
14     +--- walk_preparing_flash
15     |   +--- HAL_FLASH_Unlock
16     +--- walk_do_flash_programming
17     +--- walk_update_flash
18 Total jumps (jumpkinds): 2794

```

Printf Logs A chiusura dell'esecuzione vengono riportati i log di debug, che raccontano in modo più immediato il percorso seguito.

```

1  [ 1] **** WalkHNG Boot v 1.06 ****
2  [ 2] --> Debug usart ready...
3  [ 3] --> Start Buzzer driver
4  [ 4] --> Start TFT driver
5  [ 5] Check file WalkHNG-APP.bin
6  [ 6] Found upload file WalkHNG-APP.bin
7  [ 7] -> Load File:
8  [ 8] -> size: 0 bytes
9  [ 9] -> Code: v
10 [10] --> Update ready, Left skip, right go update

```

5.2.2 Breakpoint in memoria

Anche l'approccio a breakpoint in memoria ha raggiunto con successo il proprio criterio di terminazione: il *simulation manager* riporta uno stato in **found**, a fronte di 191 stati ancora attivi, nessuno stato scartato e nessuno stato in errore.

Rispetto all'esplorazione dei percorsi, sia il numero di stati complessivi sia il numero di salti risultano superiori riflettendo la maggior profondità raggiunta dall'esecuzione: mentre l'esplorazione si ferma a **walk_update_flash**, il breakpoint in memoria arriva fin dentro a **HAL_FLASH_Program**, la funzione che esegue effettivamente la scrittura su Flash.

Scrittura in Flash rilevata Il breakpoint sul registro **mem_write** è scattato in due occasioni, corrispondenti a due scritture consecutive di una word (4 byte) a partire dall'indirizzo 0x0800e000, corrispondente all'inizio della regione Flash applicativa:

```

1  [FLASH] Scrittura a 0x800e000, Valore: <BV32
   ↳ f_read_0x20_0x2000_0x8007d25_1060_262144[262119:262112] ..
2    f_read_0x20_0x2000_0x8007d25_1060_262144[262127:262120] ..
3    f_read_0x20_0x2000_0x8007d25_1060_262144[262135:262128] ..
4    f_read_0x20_0x2000_0x8007d25_1060_262144[262143:262136]>
5  [FLASH] Scrittura a 0x800e000, Valore: <BV32
   ↳ f_read_0x20_0x2000_0x8007d25_1062_262144[262119:262112] ..
6    f_read_0x20_0x2000_0x8007d25_1062_262144[262127:262120] ..

```

```

7   f_read_0x20_0x2000_0x8007d25_1062_262144[262135:262128] ..
8   f_read_0x20_0x2000_0x8007d25_1062_262144[262143:262136]>

```

Constraints Rispetto all'esplorazione dei percorsi (Tabella 5.1), il numero di vincoli è salito da 14 a 17: ai 14 vincoli già discussi, che fissano il contenuto del *magic number*, si aggiungono tre vincoli derivanti da una seconda invocazione di `f_stat` (indirizzo di ritorno `0x8007e01`), effettuata da `walk_program_flash` prima della scrittura effettiva. Questi vincoli restringono ulteriormente il valore di `filinfo.fsize` rispetto al primo controllo.

```

1   <Bool LShR(Reverse(f_stat_<BV32 0x800bcd2>_0x8007e01_967_704[703:672]), 0xd) != 0x0>
2   <Bool 0xffff2000 > Reverse(f_stat_<BV32 0x800bcd2>_0x8007e01_967_704[703:672])>
3   <Bool 0x0 <=s (0xa7 + (0 .. 0x80 / (0x0 .. f_stat_<BV32
   ↪ 0x800bcd2>_0x8007e01_967_704[679:672] ..
4     f_stat_<BV32 0x800bcd2>_0x8007e01_967_704[687:680] ..
5     f_stat_<BV32 0x800bcd2>_0x8007e01_967_704[695:693])[7:0]) .. 0x0) >> 0x10>

```

Call Stack

```

1   0 | walk_do_flash_programming -> HAL_FLASH_Program, returning to
   ↪ walk_do_flash_programming
2   1 | walk_program_flash -> walk_do_flash_programming, returning to walk_program_flash
3   2 | walk_update_flash -> walk_program_flash, returning to walk_update_flash
4   3 | boot_main -> walk_update_flash, returning to boot_main
5   4 | 0x0 -> 0x0, returning to 0x0

```

A differenza dell'esplorazione dei percorsi, che si ferma all'ingresso di `walk_update_flash` con uno stack di profondità 2, qui lo stack raggiunge una profondità di 5: l'esecuzione prosegue realmente all'interno del flusso di programmazione della Flash, fino a `HAL_FLASH_Program`.

Execution Path

```

1   boot_reset_handler
2   +-- boot_main
3       +-- check_sd_card
4       +-- f_open
5       +-- walk_check_update_file
6           | +-- f_stat
7           | +-- f_read
8           | +-- mod_strcmp
9           | +-- strcpy
10          | +-- strcat
11          | [...]
12      +-- walk_preparing_flash
13      +-- walk_do_flash_programming
14      +-- walk_update_flash
15          +-- f_stat
16          +-- walk_start_flash_erase
17          +-- walk_program_flash
18              +-- f_read
19              +-- printf [...] (33 chiamate)
20              +-- walk_do_flash_programming
21                  +-- HAL_FLASH_Unlock

```

```

22         +--- HAL_FLASH_Program
23         +--- FLASH_WaitForLastOperation
24 Total jumps (jumpkinds): 2938

```

```

1  [ 0] **** WalkHNG Boot v 1.06 ****
2  [...]
3  [ 4] Check file WalkHNG-APP.bin
4  [ 5] Found upload file WalkHNG-APP.bin
5  [...]
6  [10] >0x800e000-
7  [...] (33 byte, tutti 0x0)
8  [43] >2097152

```

Le voci dalla [10] alla [43] corrispondono al log emesso da `walk_program_flash` durante la scrittura: l'indirizzo di destinazione, seguito dai byte scritti (qui simbolicamente concretizzati a 0x0) e infine dalla dimensione complessiva del file, pari a 2097152 byte (2 MB) — un valore che, seppur diverso da quello osservato nell'esplorazione dei percorsi, conferma ulteriormente l'assenza di un limite realistico sulla dimensione ammessa per il file di aggiornamento (Sezione 6.2.3).

5.2.3 La vulnerabilità dell'integer underflow

I vincoli osservati sia nell'esplorazione dei percorsi ([Sezione 5.2.1](#)) sia nel breakpoint in memoria ([Sezione 5.2.2](#)) hanno mostrato che il limite superiore ammesso per `filinfo.fsize` è nell'ordine dei 0xffff2000 byte, un valore (~4 GB) del tutto incompatibile con la reale capacità della memoria Flash del dispositivo (dell'ordine del MB).

Il meccanismo Il controllo sulla dimensione del file di aggiornamento, all'interno di `walk_check_update_file`, si basa sulla variabile `available`, calcolata a partire dal registro `_flash_size_data_register`:

```

1  if (_flash_size_data_register == 0xffff) {
2      available = (uint32_t)&DAT_000f2000;
3  } else {
4      available = (_flash_size_data_register & 0xfff) * 0x400 - 0xe000;
5  }
6  if (available < filinfo.fsize) {
7      return FR_NOT_READY;
8  }

```

Il firmware non convalida il valore di `_flash_size_data_register` prima di utilizzarlo nel calcolo. Se tale registro risulta pari a 0, l'espressione $(0 \& 0xfff) * 0x400 - 0xe000$ produce un valore negativo (-0xe000); poichè `available` è dichiarata come `uint32_t`, il risultato va incontro a un *integer underflow*, avvolgendosi (*wrap-around*) al valore 0xffff2000 — un limite superiore di fatto inesistente per la dimensione del file accettato.

Verifica sperimentale La vulnerabilità è stata confermata osservando, tramite Angr, come il range di valori ammessi per `filinfo.fsize` cambi al variare del valore inizializzato in `_flash_size_data_register`:

<code>_flash_size_data_register</code>	available (atteso)	<code>filinfo.fsize</code> massimo ammesso
0x0000 (non inizializzato)	underflow	0xffff2000 (~4 GB)
0x0400 (1024 KB reali)	~968 KiB	991231 byte (~968 KiB)

Tabella 5.2: Limite superiore di `filinfo.fsize` ammesso dal solver, in funzione del valore di `_flash_size_data_register`.

Di seguito è riportato il vincolo che varia nell'esecuzione con *breakpoint in memoria*, impostando `_flash_size_data_register = 0x0400`:

```
1 <Bool 0xf2000 > Reverse(f_stat_<BV32 0x800bcd2>_0x8007e01_967_704[703:672])>
```

Con il registro correttamente inizializzato a un valore rappresentativo della memoria Flash reale (0x0400), il limite risultante è coerente con lo spazio fisicamente disponibile. Con il registro a 0 — condizione osservata quando Angr inizializza a zero i registri hardware non vincolati, ma potenzialmente riproducibile anche sul dispositivo reale in assenza di un'inizializzazione esplicita, o tramite manipolazione diretta — il controllo sulla dimensione del file risulta di fatto privo di effetto.

5.3 Vulnerabilità identificate

Sulla base dei risultati presentati nelle sezioni precedenti, è possibile riassumere le vulnerabilità individuate nel meccanismo di aggiornamento del firmware del Walk400h. Questa sezione si limita a elencarle in modo puntuale.

Validazione dell'header insufficiente Il firmware accetta come valido qualsiasi file di aggiornamento il cui contenuto inizi con la sequenza di byte determinata in [Sezione 2.2.5](#) (Tabella 5.1):

- magic header: "WalkHNG\0" (7 byte, offset 0–6);
- corpo: 20 byte, di cui solo due (offset 8 e 16) risultano effettivamente vincolati dal controllo, entrambi a valore 0x00;
- sequenza di terminazione: bf fd d1 1c (4 byte, offset 28–31).

Nessun altro byte del file, oltre a questi 11, concorre alla decisione di accettare o rifiutare l'aggiornamento.

Assenza di protezione crittografica Il meccanismo di validazione osservato non prevede alcuna forma di verifica dell'autenticità o dell'integrità del file, in particolare:

- non è presente una firma digitale, che consentirebbe di verificare che il file provenga da una fonte autorizzata;
- non è presente un meccanismo di checksum o hash, che consentirebbe di rilevare corruzioni o modifiche del file.

Controllo limitato sul nome del file Il controllo sul nome del file di aggiornamento, osservato nell’analisi del bootloader ([Sezione 3.3](#)), si limita alla corrispondenza esatta con la stringa `WalkHNG-APP.bin`, senza ulteriori verifiche sul percorso o sull’origine del file.

Assenza di validazione su `_flash_size_data_register` Come dimostrato in [Sezione 5.2.3](#), il valore del registro `_flash_size_data_register`, utilizzato per calcolare lo spazio disponibile in Flash (`available`), non viene validato prima dell’uso. Un valore pari a zero genera un integer underflow che porta `available` ad assumere un valore prossimo al massimo rappresentabile per un intero a 32 bit (`0xffff2000`, ~ 4 GB), rendendo di fatto inefficace il controllo sulla dimensione del file di aggiornamento.

Capitolo 6

Conclusioni

6.1 Efficacia e limiti degli strumenti utilizzati

6.1.1 Limitazioni RetDec + KLEE

Gli esiti presentati in ?? permettono di trarre alcune conclusioni sull'adeguatezza della pipeline RetDec + Klee per l'analisi di sicurezza di firmware embedded reali.

Il problema piú rilevante riscontrato non è di natura computazionale, ma di **fedeltá di decompilazione**: RetDec non ha prodotto un codice funzionalmente equivalente al binario originale. Come dimostrato in [Sezione 5.1.2](#), l'intero ramo di validazione del magic number risulta mancante. Si tratta di un problema strutturale: qualunque analisi condotta a valle di una compilazione infedele eredita la stessa incompletezza, e non è possibile escludere che ci omissioni analoghe siano presenti anche in altre parti del codice

A questo si aggiungono altre limitazioni strettamente legate a KLEE: anche assumendo una decompilazione fedele, l'applicazione di KLEE su un firmware bare-metal resta un processo oneroso. Da un lato, l'assenza di un modello dell'hardware sottostante richiede una dipendenza massiccia da stub, i quali devono essere scritti e mantenuti manualmente per ogni periferica o funzione non modellabile. Dall'altro, l'esecuzione stessa ha richiesto una manipolazione estensiva del codice LLVM IR prodotto da RetDec, per renderlo assemblabile ed eseguibile ([Sezione 4.1](#)). Entrambi gli interventi allontanano progressivamente il codice dal comportamento reale del firmware, rendendo meno affidabili i risultati.

Nonostante queste limitazioni, è opportuno segnalare che la pipeline non è stata del tutto inutile: l'esecuzione è comunque riuscita a progredire fino all'ingresso di `walk_check_update_file`, superando correttamente le fasi di inizializzazione hardware grazie agli stub. Questo risultato parziale, per quanto lontano dall'obiettivo di un'analisi completa della logica di validazione, conferma che l'approccio non è concettualmente inapplicabile, ma che richiede un investimento sproporzionato di tempo rispetto al beneficio ottenuto.

6.1.2 Efficacia e limiti dell'approccio con Angr

I risultati ottenuti con Angr, presentati nelle [Sezione 5.1–Sezione 5.2](#), permettono un confronto diretto con la pipeline RetDec + KLEE discussa in precedenza, e restituiscono un bilancio nettamente piú positivo, pur non privo di limitazioni proprie.

Il vantaggio principale osservato è diretto: lavorando sul binario senza un passaggio intermedio di decompilazione, Angr non eredita il problema di fedeltá che ha compromesso

l'approccio precedente. L'`execution path` ricostruito in [Sezione 5.2](#) include infatti il ramo completo di validazione del magic number.

Ciò non significa che l'approccio sia stato privo di difficoltà. Anche con Angr, un'esecuzione simbolica priva di guida (l'approccio "blind" discusso in [Sezione 4.2.1](#)) si è rivelata impraticabile, bloccandosi nei medesimi cicli di inizializzazione hardware che affliggevano KLEE, per la stessa ragione di fondo: l'assenza di un modello realistico delle periferiche. La differenza rispetto a KLEE non è quindi l'assenza di lavoro manuale, ma la sua natura: mentre con KLEE tale lavoro consisteva in grande parte in interventi correttivi sul codice intermedio, con Angr si è tradotto principalmente nella definizione di stub comportamentali, un'attività direttamente più collegata alla logica del firmware che agli strumenti utilizzati.

Questo contrasto è visibile anche a un livello più tecnico: mentre in Angr è stato possibile scrivere direttamente nel buffer di destinazione di `f_read` tramite l'API di memoria dello stato simbolico, lo stesso obiettivo con KLEE ha richiesto di aggirare un vincolo strutturale — l'impossibilità di rendere simbolico direttamente un indirizzo nello spazio di memoria del programma — tramite riscrittura manuale del codice intermedio ([Sezione 4.1.3](#)).

Infine, il confronto tra i due approcci basati su Angr (esplorazione dei percorsi e breakpoint in memoria) evidenzia un compromesso tra ampiezza e profondità: l'esplorazione dei percorsi verifica in modo più rapido ed economico la sola raggiungibilità di un target, mentre il breakpoint in memoria, lasciando eseguire realmente il codice, offre un'osservazione più completa del comportamento del firmware - inclusi eventuali effetti collaterali - al costo di un'esplorazione più profonda e onerosa. Nessuno dei due approcci è preferibile in assoluto: la scelta dipende dall'obiettivo specifico dell'analisi.

6.1.3 Sintesi comparativa

Il confronto condotto in questo capitolo permette di individuare in Angr lo strumento più adatto al contesto del firmware embedded analizzato, sia per ragioni strutturali che di comodità d'uso. Operando direttamente sul binario, Angr non introduce distorsioni in fase di decompilazione, e permette di intervenire sul modello della memoria dello stato simbolico attraverso un'API diretta, senza modifiche manuali del codice intermedio. La sua natura di libreria Python ha inoltre facilitato l'implementazione di meccanismi di osservabilità personalizzati - la cattura dei messaggi di debug e il breakpoint sulla scrittura in Flash (Capitolo 4) - risultati determinanti per l'interpretazione dei risultati ottenuti nel Capitolo 6.

6.2 Impatto di sicurezza delle vulnerabilità identificate

6.2.1 Scenario di attacco combinato

Le vulnerabilità scritte in [Sezione 5.3](#), seppure singolarmente limitate, si combinano in una catena che permette ad un attaccante con accesso fisico temporaneo alla scheda SD del dispositivo di installare firmware arbitrario e non verificato

1. **Preparazione del file.** L'attaccante prepara un file chiamato esattamente `WalkHNG-APP.bin`

2. **Costruzione dell'header.** L'header del file deve essere composto da: 7 byte del magic "WalkHNG", i 4 byte di terminazione `bf fd d1 1c` e il primo e l'ultimo byte del padding devono essere `0x00`. I restanti byte dell'header e il payload sono liberi.
3. **Assenza di verifica di autenticità.** Superati i controlli precedenti, il firmware non effettua alcuna verifica sul contenuto del file.
4. **Scrittura su flash.** Superate le verifiche, il firmware procede con la scrittura della memoria Flash senza ulteriori controlli.

A questa catena si aggiunge, come fattore potenzialmente aggravante, la vulnerabilità dell'integer underflow su `_flash_size_data_register` (Sezione 5.2.3): se un attaccante fosse in grado di influenzare tale registro, il limite sulla dimensione del file risulterebbe di fatto rimosso. Non è stato individuato, nell'ambito di questo lavoro, un meccanismo concreto per farlo dall'esterno; resta quindi una vulnerabilità dimostrata a livello di modello simbolico, la cui sfruttabilità pratica richiede ulteriore verifica.

L'esito di questa catena di attacco è la possibilità di installare sul dispositivo, senza autorizzazione, un firmware arbitrario scelto dall'attaccante - con conseguenze che vanno dalla compromissione persistente del dispositivo fino, nella peggiore delle ipotesi, all'alterazione del suo comportamento clinico osservabile.

6.3 Sviluppi futuri

L'analisi condotta in questa tesi si è concentrata sul bootloader per la sua criticità. Il firmware applicativo, eseguito dopo il trasferimento del controllo a `run_app`, non è stato oggetto di analisi e potrebbe contenere ulteriori vulnerabilità, ad esempio nella gestione dei dati acquisiti o nelle altre interfacce del dispositivo. Si potrebbe inoltre verificare come il dispositivo interagisca con l'applicazione sul terminale, alla ricerca di ulteriori possibili vettori di attacco.

Buona parte del lavoro manuale descritto nel Capitolo 4 (identificazione delle funzioni da stubbare, scrittura dei singoli hook) potrebbe essere parzialmente automatizzata a partire dai risultati dell'analisi statica. In assenza di una tabella dei simboli, tale automazione non potrebbe basarsi sui nomi delle funzioni, ma su tecniche di riconoscimento per pattern del codice compilato: ad esempio, database di *Function ID* (già supportati da Ghidra) permettono di identificare funzioni di libreria note, come le routine HAL, tramite un confronto a livello di istruzioni macchina anziché di simboli. In alternativa, euristiche più semplici — come la ricerca di cicli di polling su indirizzi MMIO con timeout, pattern osservato manualmente in questo lavoro (Sezione 4.2.3) — potrebbero proporre automaticamente candidati da stubbare, riducendo l'intervento manuale necessario per adattare l'analisi a un nuovo target.

Poiché le vulnerabilità sono state dimostrate solamente a livello simbolico, un naturale passo successivo consiste nel verificarne la sfruttabilità concreta su un dispositivo reale. Questo è di interesse in particolare per l'aspetto rimasto irrisolto in questo lavoro, ovvero l'identificazione di un meccanismo pratico per influenzare `_flash_size_data_register` (Sezione 5.2.3), così da convalidare sul campo quanto osservato nel modello.

6.4 Considerazioni finali

Gli obiettivi prefissati per questa tesi possono ritenersi raggiunti: è stato individuato uno strumento di analisi in grado di operare efficacemente su firmware bare-metal, ed è stato possibile applicare e confrontare metodologie di analisi differenti sullo stesso caso di studio.

L'esperienza condotta permette di affermare che l'esecuzione di test di sicurezza su firmware embedded è un obiettivo raggiungibile, ma tutt'altro che immediato: richiede, come mostrato nei capitoli precedenti, un lavoro non trascurabile di adattamento degli strumenti al contesto specifico, spesso a partire da un'analisi manuale preliminare del target.

Le vulnerabilità individuate nel firmware del Walk400h, più che derivare da errori di implementazione isolati, riflettono per lo più l'assenza di politiche di sicurezza strutturate nel processo di aggiornamento — una scelta verosimilmente legata ai vincoli tipici dei dispositivi embedded, dove risorse computazionali e di memoria limitate rendono l'adozione di meccanismi crittografici un costo non sempre ritenuto giustificabile. Nel contesto di un dispositivo medicale, tuttavia, questo compromesso merita una riflessione più attenta: la possibilità di installare firmware non verificato rappresenta un rischio che va ben oltre la sola integrità del dispositivo, coinvolgendo la sicurezza del paziente che ne fa uso.

Bibliografia

- [1] Erika Ábrahám, József Kovács e Anne Remke. “SMT: Something You Must Try”. In: *Integrated Formal Methods (iFM 2023)*. Vol. 14300. Lecture Notes in Computer Science. Springer, 2023, pp. 3–18. DOI: [10.1007/978-3-031-47705-8_1](https://doi.org/10.1007/978-3-031-47705-8_1).
- [2] Agenzia per la Cybersicurezza Nazionale. *Sanità nel mirino: in forte aumento gli incidenti informatici nelle strutture italiane*. 2025. URL: <https://www.acn.gov.it/portale/w/sanita-nel-mirino-in-forte-aumento-gli-incidenti-informatici-nelle-strutture-italiane> (visitato il giorno 2026-07-16).
- [3] Giovanni Alberici. “Elaborato del Tirocinio Triennale sul Walk400h”.
- [4] Avast Software. *RetDec: A Retargetable Machine-Code Decompiler*. URL: <https://github.com/avast/retdec> (visitato il giorno 2026-07-16).
- [5] Roberto Baldoni et al. “A Survey of Symbolic Execution Techniques”. In: *ACM Computing Surveys* 51.3 (2018), 50:1–50:39. DOI: [10.1145/3182657](https://doi.org/10.1145/3182657). URL: <https://www.diag.uniroma1.it/delia/papers/csur18.pdf>.
- [6] Cristian Cadar e Koushik Sen. “Symbolic Execution for Software Testing: Three Decades Later”. In: *Communications of the ACM* 56.2 (2013), pp. 82–90. DOI: [10.1145/2408776.2408795](https://doi.org/10.1145/2408776.2408795). URL: <https://dl.acm.org/doi/epdf/10.1145/2408776.2408795>.
- [7] Sang Kil Cha et al. “Unleashing Mayhem on Binary Code”. In: *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 380–394. DOI: [10.1109/SP.2012.31](https://doi.org/10.1109/SP.2012.31). URL: <https://doi.org/10.1109/SP.2012.31>.
- [8] Vitaly Chipounov, Volodymyr Kuznetsov e George Candea. “S2E: A Platform for In-Vivo Multi-Path Analysis of Software Systems”. In: *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: ACM, 2011, pp. 265–278. DOI: [10.1145/1961296.1950396](https://doi.org/10.1145/1961296.1950396). URL: <https://doi.org/10.1145/1961296.1950396>.
- [9] Drew Davidson et al. “FIE on Firmware: Finding Vulnerabilities in Embedded Systems Using Symbolic Execution”. In: *22nd USENIX Security Symposium (USENIX Security 13)*. Washington, D.C.: USENIX Association, 2013, pp. 463–478. ISBN: 978-1-931971-03-4. URL: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/davidson>.
- [10] Daniel Halperin et al. “Pacemakers and Implantable Cardiac Defibrillators: Software Radio Attacks and Zero-Power Defenses”. In: *Proceedings of the 2008 IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2008, pp. 129–142. DOI: [10.1109/SP.2008.31](https://doi.org/10.1109/SP.2008.31).
- [11] KLEE Project. *KLEE Symbolic Execution Engine*. URL: <https://klee-se.org/> (visitato il giorno 2026-07-16).

- [12] Chris Lattner e Vikram Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation”. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86. DOI: [10.1109/CGO.2004.1281665](https://doi.org/10.1109/CGO.2004.1281665). URL: <https://doi.org/10.1109/CGO.2004.1281665>.
- [13] National Security Agency. *Ghidra: A Software Reverse Engineering (SRE) Framework*. URL: <https://github.com/NationalSecurityAgency/ghidra> (visitato il giorno 2026-07-16).
- [14] Abdullah Qasem et al. “Automatic Vulnerability Detection in Embedded Devices and Firmware: Survey and Layered Taxonomies”. In: *ACM Computing Surveys* 54.2 (2021), 25:1–25:42. DOI: [10.1145/3432893](https://doi.org/10.1145/3432893). URL: <https://doi.org/10.1145/3432893>.